UNIVERSITY OF PLYMOUTH

COMP3004 Advanced Computing and Networking Infrastructures

Assessment 1: Testing NGINX for Load Balancing Docker Compose containers

Reece Davies [10572794]

BSc (Hons) Computer Science [G407]

Table of Contents

Introduction

In current standards, developing and deploying modern web applications can be a daunting task, especially when under strict conditions or requirements; however, with the use of "Docker" or other Platform-as-a-Service systems, this can be easily overcome. Docker enables one to create an environment where they are easily able to manage their applications. This report explains the development of Docker containers for multiple web-servers and automating the process with Dockerfile and Docker Compose configuration. Furthermore, testing will be conducted on the Docker containers for load balancing with NGINX, which in turn will improve the performance of the hypothetical business's webserver.

System Setup

The entirety of this project will be conducted on a Linux Virtual Machine using "Ubuntu Linux Server 16.04.5", which is accessed through "VM Workstation". As all the testing was conducted from the host computer, the Linux Server's network connection must be set to "bridged". By doing so, the virtual machine is automatically connected to the host computer's network; this can be achieved through "Virtual Machine Settings" in VM Workstation.

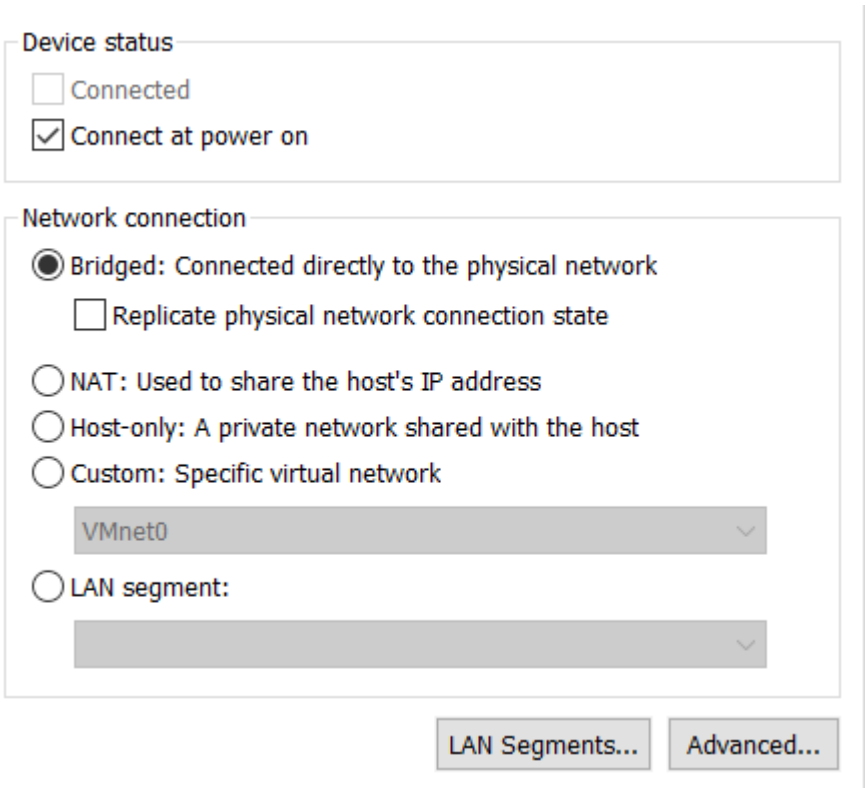

Figure 1

Once the virtual machine has been setup, numerous Linux packages must be installed onto the server, as these are required to successfully run the relevant software used in this demonstration. The Linux installation commands are as follows:

Command	Package Installed	Explanation
$sudo apt-get install openssh-server	Open SSH Server	Allows the user to remotely connect to the virtual machine through Secure Shell.
$sudo apt-get install apache2	Apache server	HTTP web server software to power the Ubuntu Server VM.
$sudo apt-get install nginx	NGINX	HTTP web server, which is used for load balancing in this demonstration.
$sudo apt-get install docker.io	Docker	Platform as a Service (PAAS) to virtualise and distribute software in packages, referred to as 'containers'.
$sudo apt install docker-compose	Docker Compose	Tool to automate and run multi-container Docker applications.

After the packages have successfully been installed onto the Linux Server, the user will then be able to set up the appropriate files for Docker Compose and relevant processes. Everything has been saved in the new directory "webserver", and each file serves an important role in the automation process of creating the Docker Images, Docker Containers, and then automatically load balancing all incoming traffic. The contents have been assembled in the following structure:

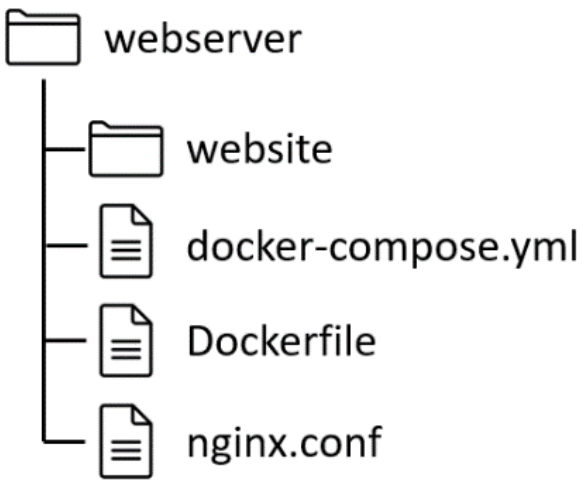

Figure 2

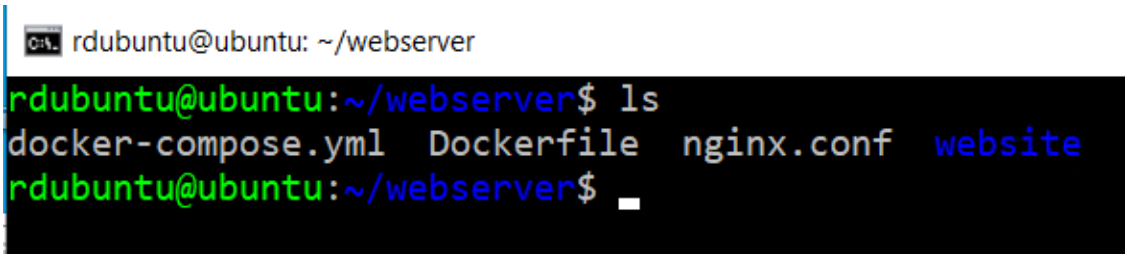

Figure 3

- The "website" directory holds all the relevant files required for the business's ecommerce application.
- "docker-compose.yml" is a YAML file required to run Docker Compose.
- "Dockerfile" is responsible for creating and modifying new Docker Images
- "nginx.conf" is an NGINX configuration file that is set up to allow for load balancing.

Automation Process

This demonstration has been configured to make use of Dockerfile and Docker Compose to automate the tedious process of creating Docker Images and then assigning them to new Docker containers. Firstly, Dockerfile is a simple text file that contains the commands to create and potentially modify new Docker images. The contents of the Dockerfile are displayed in figure 4.

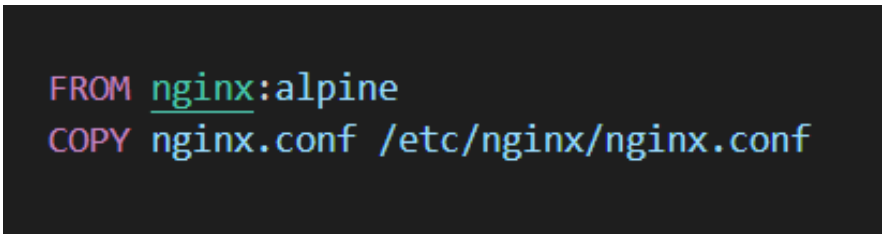

Figure 4

This Dockerfile creates the reverse proxy image, and uses the "nginx.conf" file after copying it to the proxy container. The command "FROM" imports the specified Docker image, which is then used as the base for the rest of the Dockerfile. Subsequently, "COPY" is used to add directories and files to the selected Docker image; in this case, the NGINX configuration file is added for the additional features required to implement load balancing.

Once Dockerfile has been configured properly, Docker Compose will be used as a tool for automating the process of reading the relevant images from the Dockerfile and creating new Docker containers for the web server. It allows the user to easily create new containers with a single command, thus preventing inconsistencies or runtime errors that may occur when done manually. The contents of the "docker-compose.yml" file are displayed in figure 5.

```yaml
version: "2"

services:
  db:
    image: mysql:latest
    environment:
      - MYSQL_ROOT_PASSWORD=abcd1234
    restart: always
    volumes:
      - db_vol:/var/lib/mysql

  ecommercesite:
    image: nginx:latest
    restart: always

    volumes:
      - ./website:/usr/share/nginx/html/

    depends_on:
      - db
    links:
      - db

  nginx:
    image: nginx:latest
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - ecommercesite
    ports:
      - "4000:4000"

volumes:
  db_vol:
```

Figure 5

Within the Docker Compose file, three unique services have been defined: “db”, “ecommercesite”, and “nginx”. Firstly, the “db” service is specifically for the website’s database, as it uses the “mysql:latest” image to build the container. By defining “db_vol”, it therefore binds the directory “/var/lib/mysql”, which means it ensures that any persisting data generated and used by Docker containers are stored in this path.

The “ecommercesite” service will behave as the worker(s) for the webserver; there will be numerous containers for this service, each being responsible for fetching the webserver’s content . Furthermore, what the client sees is taken from this service’s “volumes” key, which maps the contents of the “website” directory to the path “/usr/share/nginx/html”. In addition, by making this service depend on “db”, it therefore prevents the database from loading if this service has not been loaded.

Lastly, the “nginx” service is used as the load balancer for the webserver, alongside with the “nginx.conf” file. This service will be loaded from port 4000 and then transfer the incoming traffic to an “ecommercesite” container at default port 80. If “ecommercesite” required a different port, the key “expose” would be added, alongside the desired destination port.

With regards to the term ‘load balancing’, when numerous containers have been deployed throughout a collection of webservers, load balancers are responsible for allowing the different Docker containers to be accessible via the same host port. If a company’s webserver is not able to handle its current network traffic, load balancing would prove beneficial as it distributes the traffic to the healthiest and most available Docker container. To implement load balancing to this solution, the “nginx.conf” file is created in the same directory as the “docker-compose.yml” file.

```
user  nginx;

events {
    worker_connections   1000;
}
http {
        server {
            listen 4000;
            location / {
              proxy_pass http://ecommercesite:80;
            }
        }
}
```

Figure 6

As a result, this file adds configuration functions to NGINX to listen at port 4000, and forward the request to “http://ecommercesite:80”, resulting in the proxy server to load one of the containers from the “ecommercesite” service. If this service was exposed from a different port in the “docker-compose.yml” file, those changes must also be reflected here.

Discussion of Results

With all the configuration files set up, the user is then able to easily launch the Docker containers, alongside the NGINX load balancer. Firstly, the shell command "scale" must be executed, as displayed in figure 7. This command will start three instances of the "ecommercesite" service, resulting in three separate ecommercesite 'worker' containers.

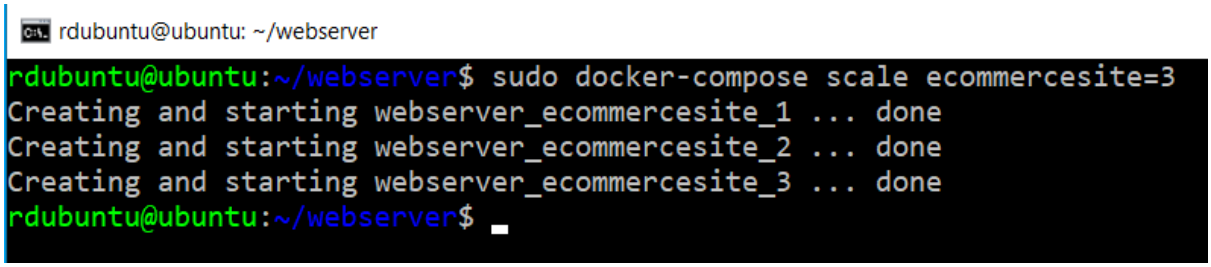

Figure 7

Subsequently the user must start Docker Compose with the "up" command, alongside appropriate options. This command builds (or re-builds) and starts the containers attached to each service, as displayed in figure 8; furthermore, the option "-d" is added to run the containers in the background. Figure 9 also establishes that the containers have successfully been created and are running.

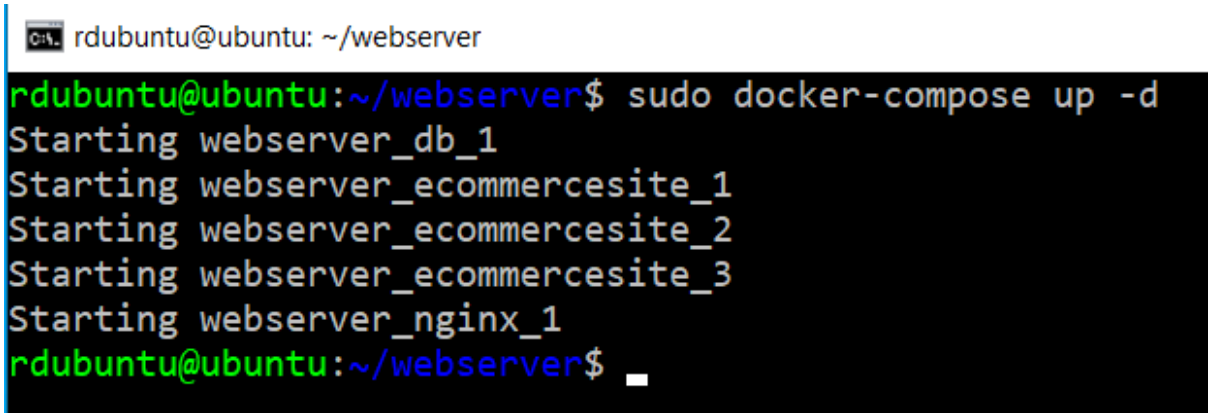

Figure 8

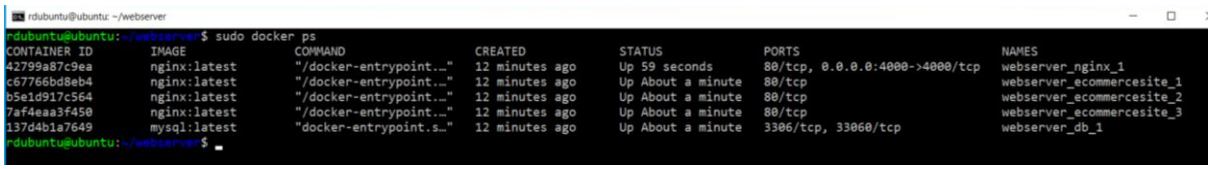

Figure 9

Moreover, figure 10 demonstrates this command being executed without ‘detach’ mode enabled, and therefore displays runtime logs. When the webpage is loaded (displayed in figure 11), the command line outputs a log for which container is assigned to the specified client; as a result, “nginx” loads one of the following containers:

- webserver_ecommercesite_1
- webserver_ecommercesite_2
- webserver_ecommercesite_3

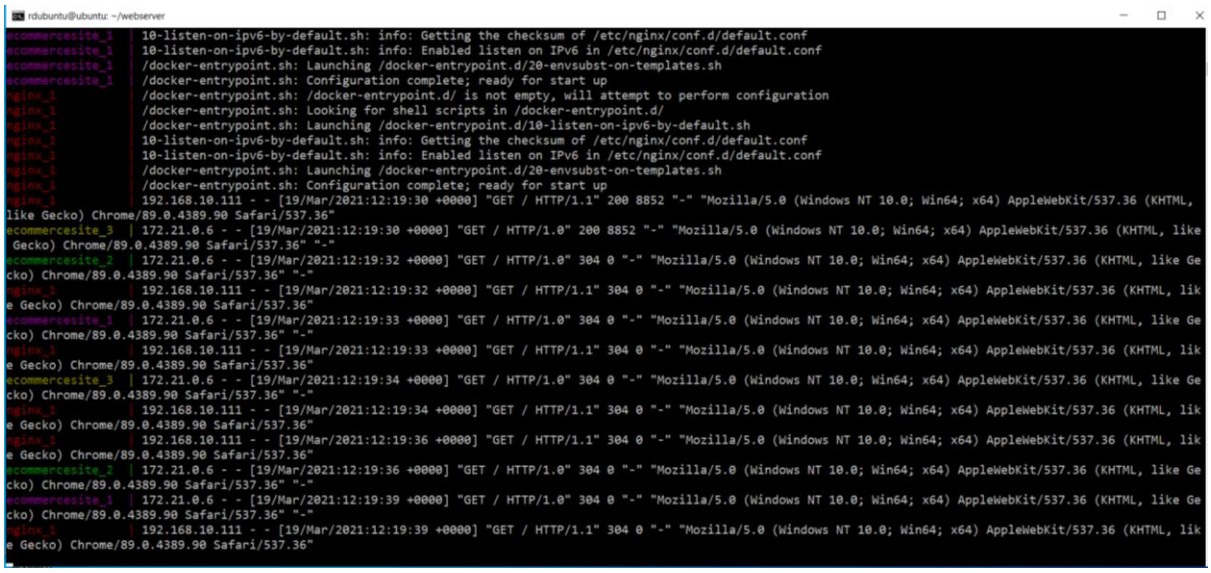

Figure 10

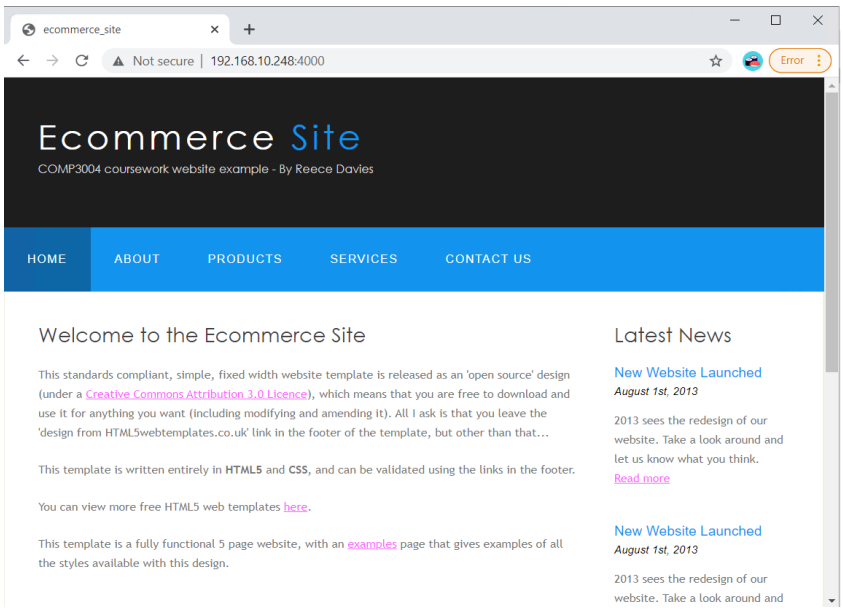

Figure 11

Of course, by implementing the NGINX load balancer to the business's website, this will improve the efficiency of its web-servers; however, this will also be dependent upon the physical servers that host Docker and NGINX. If the incoming traffic significantly increases beyond normal conditions, the server may not be able to handle the quantity of clients attempting to access the webpage, regardless of the number of worker containers. It is therefore apparent that this setup may require additional adjustments to improve performance if at a larger scale. For instance, this solution could potentially be configured to make use of a collection of physical servers, with each server hosting numerous Docker containers, which will result in the dispersion of the required computing power.

Additionally, a slightly different method of approach could have been taken for load balancing separate Docker containers with NGINX. The containers would all be declared with a unique name and port, however all linked to the website's contents in their "volumes". The "nginx.conf" file would then be adjusted to have "upstream loadbalancer" directed to the different Docker containers and their designated port. These adjusted configuration files can be seen in figures 12 & 13. A demonstration of this is shown in figure 14, where each container consists of a slightly different HTML page, as declared in their "volumes" key.

```yaml
version: "2"

services:
  ecommercesite1:
    image: nginx:latest
    restart: always
    volumes:
      - ./websites/container1:/usr/share/nginx/html/
    ports:
      - "5001:5000"

  ecommercesite2:
    image: nginx:latest
    restart: always
    volumes:
      - ./websites/container2:/usr/share/nginx/html/
    ports:
      - "5002:5000"

  ecommercesite3:
    image: nginx:latest
    restart: always
    volumes:
      - ./websites/container3:/usr/share/nginx/html/
    ports:
      - "5003:5000"

  nginx:
    image: nginx:latest
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - ecommercesite1
      - ecommercesite2
      - ecommercesite3
    ports:
      - "8080:80"
```

Figure 12

```
user  nginx;

events {
    worker_connections   1000;
}
http {
        upstream loadbalancer {
          server 192.168.10.248:5001;
          server 192.168.10.248:5002;
          server 192.168.10.248:5003;
        }
        server {
              listen 80;
              location / {
                proxy_pass http://loadbalancer;
              }
        }
}
```

Figure 13

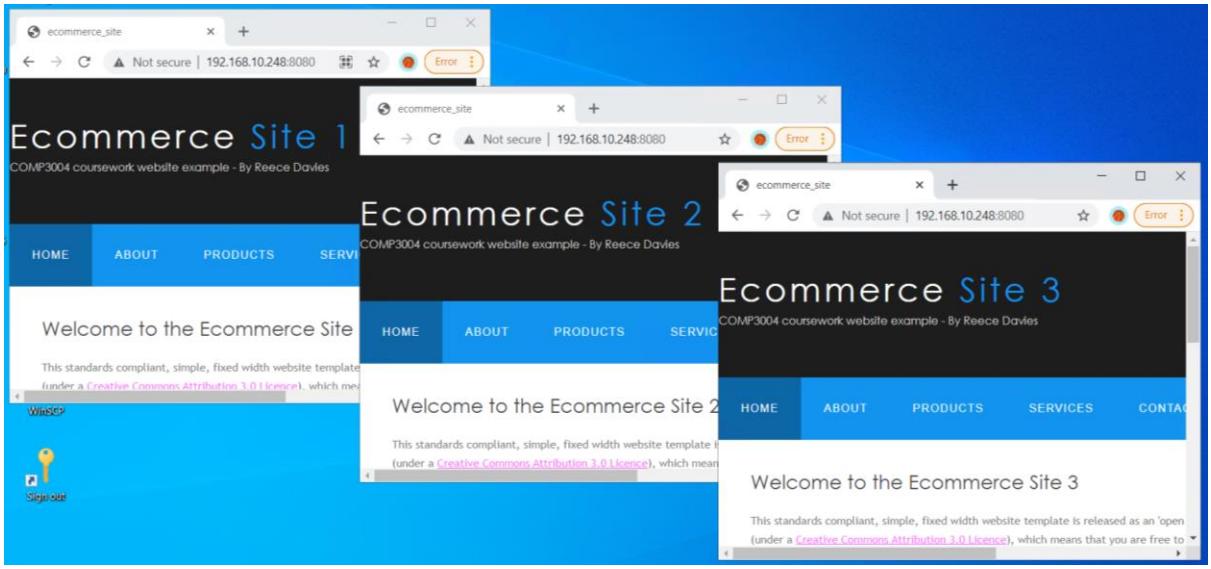

Figure 14

Conclusion

This document has demonstrated the use of NGINX for load balancing numerous Docker containers, and how this may be beneficial to an e-commerce business that cannot handle their current traffic. Nevertheless, the approach I have taken allows the user to set the scale of the Docker service, thus improving automation as the number of instances can easily be increased or decreased based on specific conditions. Regardless of approach, Docker proves to be an important and cost-effective tool for efficiently deploying and managing modern applications or servers.

References

Avi Networks. (2019). Container Load Balancing Definition. Viewed 5th Mar 2021. Avi Networks. <https://avinetworks.com/glossary/container-load-balancing/>.

Collabnix. (2020). Difference between Docker Compose Vs Dockerfile. Viewed 26th Feb 2021. Docker Labs. <https://dockerlabs.collabnix.com/beginners/difference-compose-dockerfile.html>.

Docker. (2021). Overview of Docker Compose. Viewed 5th Mar 2021. Docker. <https://docs.docker.com/compose/>.

OUASSINI, A. (2020). Sample Load balancing solution with Docker and Nginx. Viewed 8th Mar 2021. Towards Data Science. <https://towardsdatascience.com/sample-load-balancing-solution-with-docker-and-nginx-cf1ffc60e644>.

Pavlovic, R. Atkins, G. (2020). Docker: Top 7 Benefits of Containerization. Viewed 18th Mar 2021. Hentsu. <https://hentsu.com/docker-containers-top-7-benefits/>.

Rane, V. (2020). Microservices: Scaling and Load Balancing using docker compose. Viewed 8th Mar 2021. Medium. <https://medium.com/@vinodkrane/microservices-scaling-and-load-balancing-using-docker-compose-78bf8dc04da9>.

Shamun, I. (2020). Docker Compose: The Perfect Development Environment. Viewed 20th Mar 2021. Daily.dev. <https://daily.dev/blog/docker-compose-the-perfect-development-environment>.